MINI PROJECT

SINSHA C
21/05/2026

AWS Multi-region VPC Peering and Network Setup.

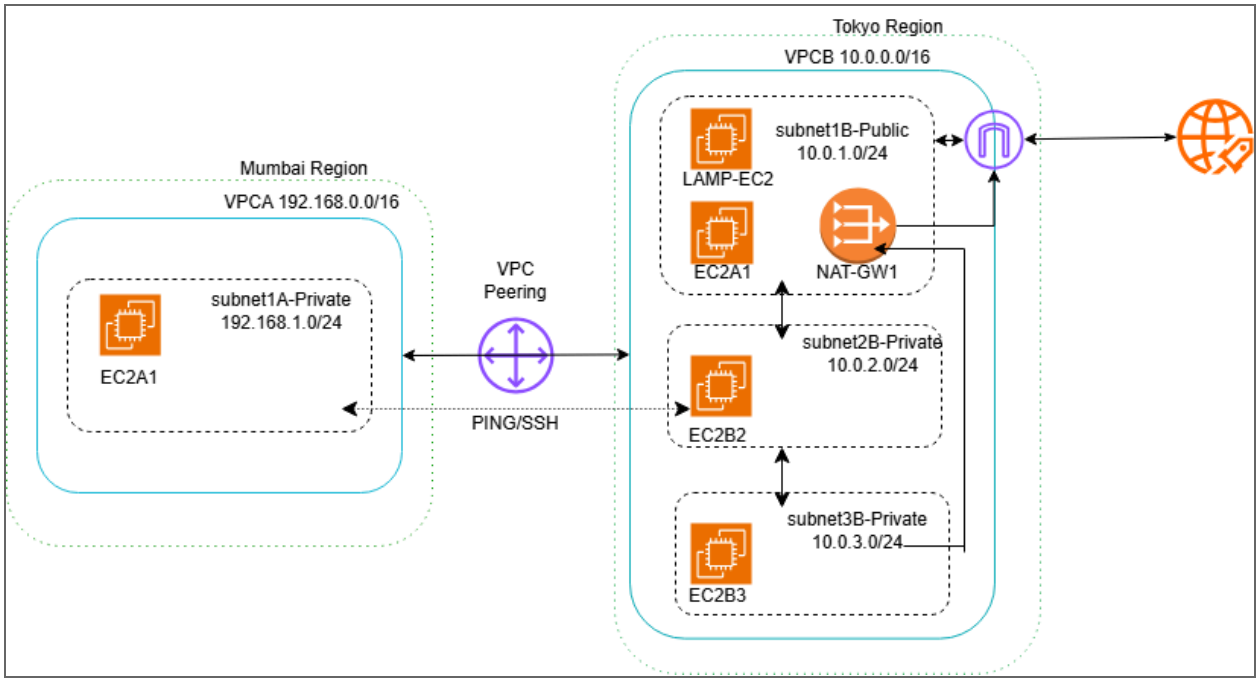
Overview

This project demonstrates how to build a secure multi-region architecture using Amazon VPC, subnets, EC2 instances, and VPC Peering across two AWS regions: Mumbai and Tokyo.

We will design:

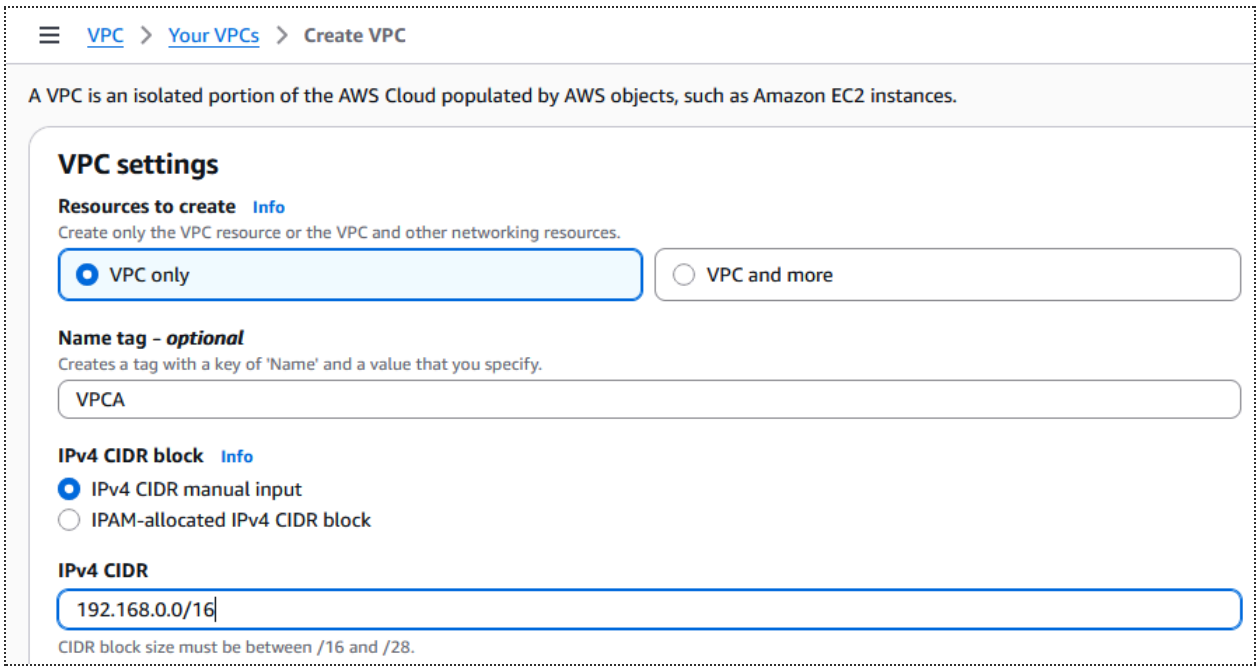
- VPC A (Mumbai) → Private workload
- VPC B (Tokyo) → Public + Private segmented architecture
- Cross-region VPC Peering
- Controlled internet access using NAT Gateway
- EC2 instances in each subnet (including LAMP stack server)

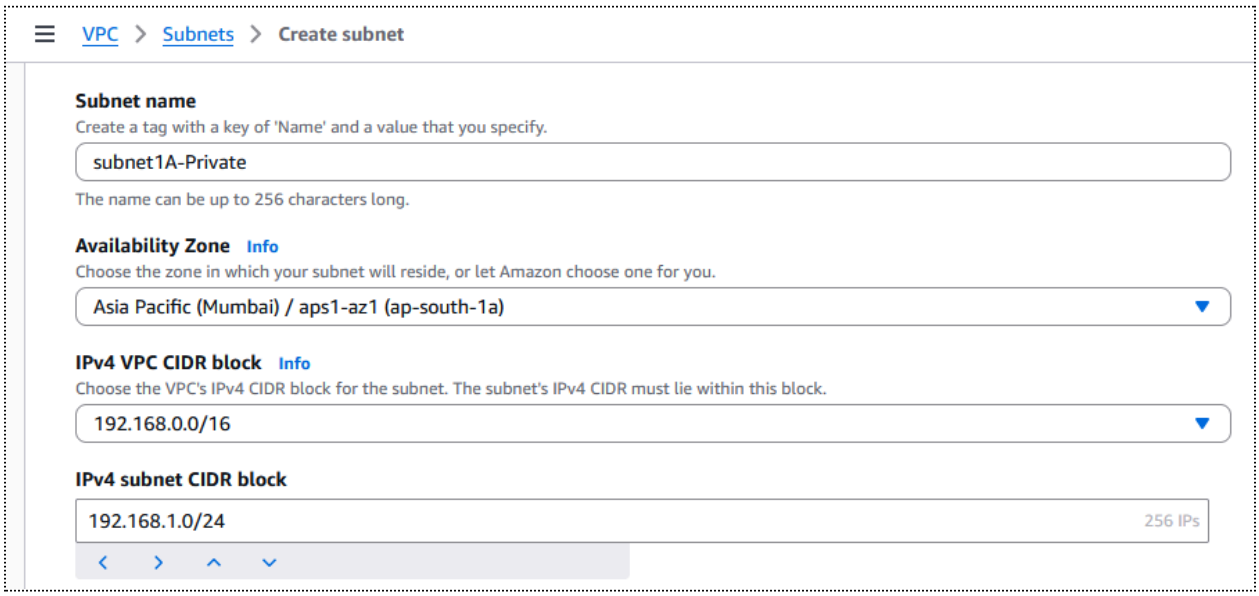
Architecture diagram

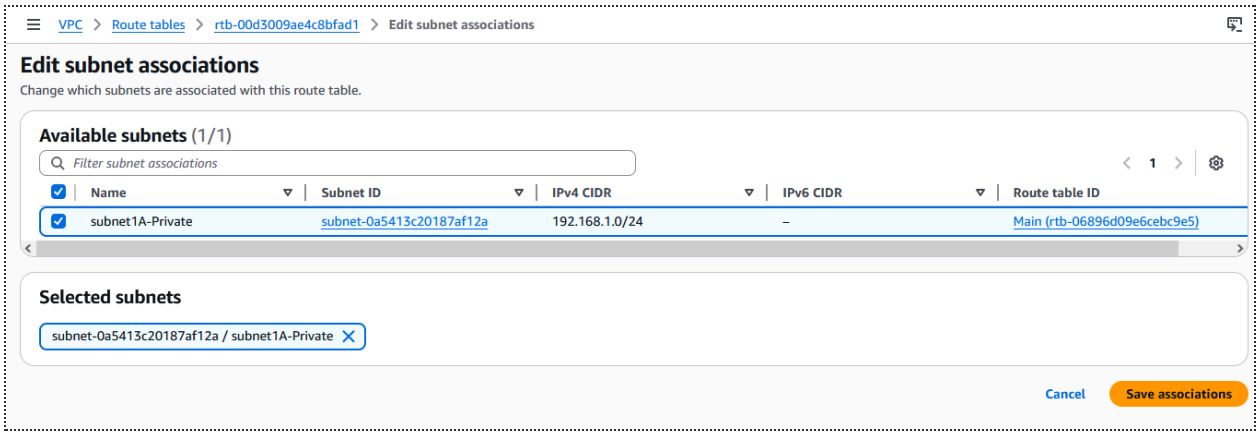


Step 1: Create VPCs

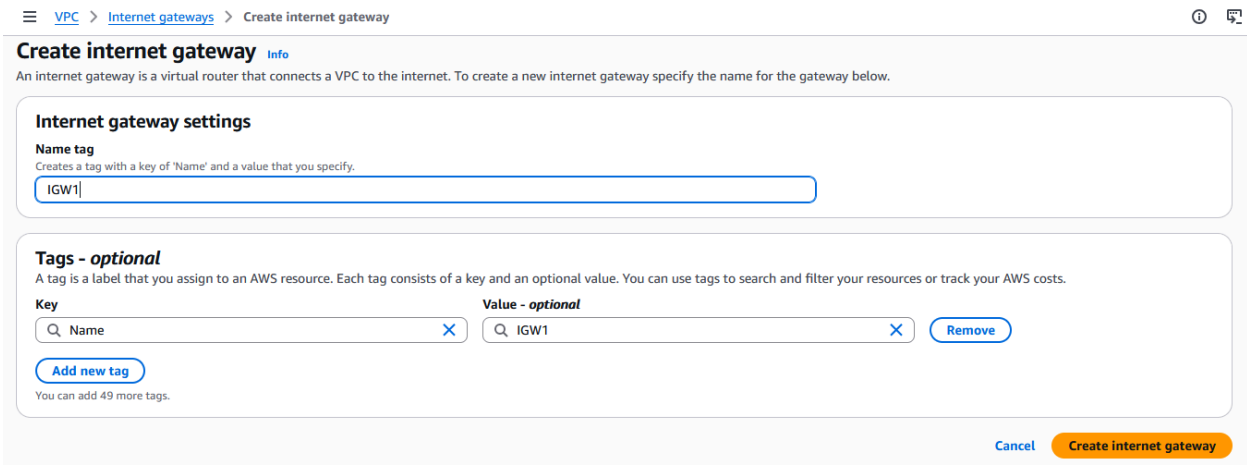
- [ ] Create VPC A (Mumbai)
  - [ ] Login to AWS and select the service VPC.
  - [ ] Create a VPC named VPCA with CIDR 192.168.0.0/16
  - [ ] Create subnet1A-Private with CIDR 192.168.1.1/24
  - [ ] Create Route table RTA and associate with subnet1A-Private


VPC > Your VPCs > Create VPC
A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.
VPC settings
Resources to create Info
Create only the VPC resource or the VPC and other networking resources.
VPC only
VPC and more
Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.
VPCA
IPv4 CIDR block Info
IPv4 CIDR manual input
IPAM-allocated IPv4 CIDR block
IPv4 CIDR
192.168.0.0/16
CIDR block size must be between /16 and /28.

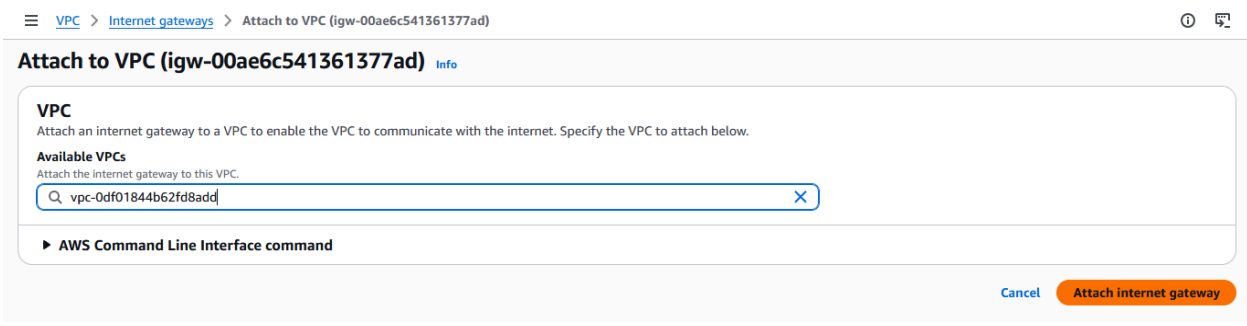

VPC > Subnets > Create subnet
Subnet name
Create a tag with a key of 'Name' and a value that you specify.
subnet1A-Private
The name can be up to 256 characters long.
Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
Asia Pacific (Mumbai) / aps1-az1 (ap-south-1a)
IPv4 VPC CIDR block Info
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
192.168.0.0/16
IPv4 subnet CIDR block
192.168.1.0/24
256 IPs



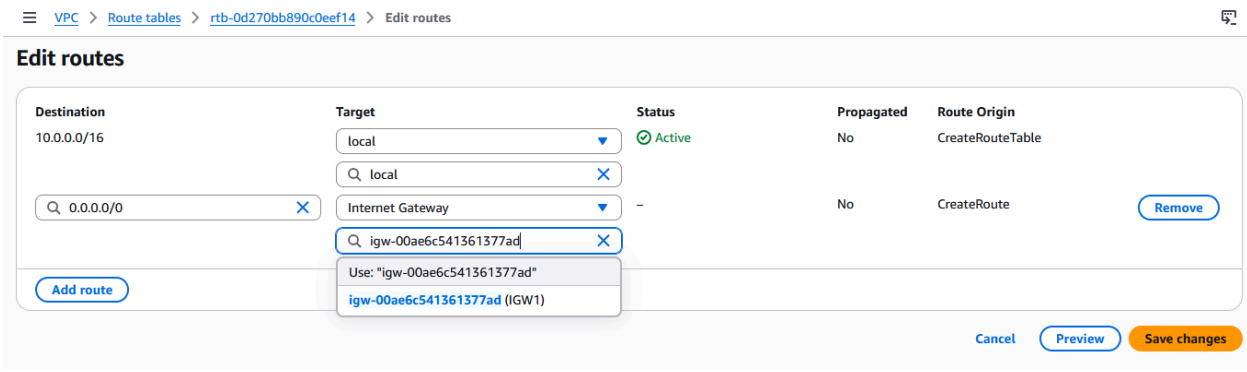
- [ ] Create VPC B (Tokyo)
  - [ ] Change the region to Tokyo and create another VPC named VPCB with CIDR 10.0.0.0/16
  - [ ] Create subnets
    - [ ] subnet1B-Public with CIDR 10.0.1.0/24
    - [ ] subnet2B-Private with CIDR 10.0.2.0/24
    - [ ] subnet3B-Private with CIDR 10.0.3.0/24
  - [ ] Create corresponding route table and associate with subnets RTB1,RTB2 and RTB3
  - [ ] Create Internet gateway IGW1



- [ ] Attach IGW1 to VPCB



- [ ] Go to the route table associated with subnet1B-Public. Edit route and add igw id as route



Step 2: Launch EC2 Instances

Prerequisites

- [ ] Login to both region and create keypair
- [ ] Create security group which allow ssh, ICMP and HTTP

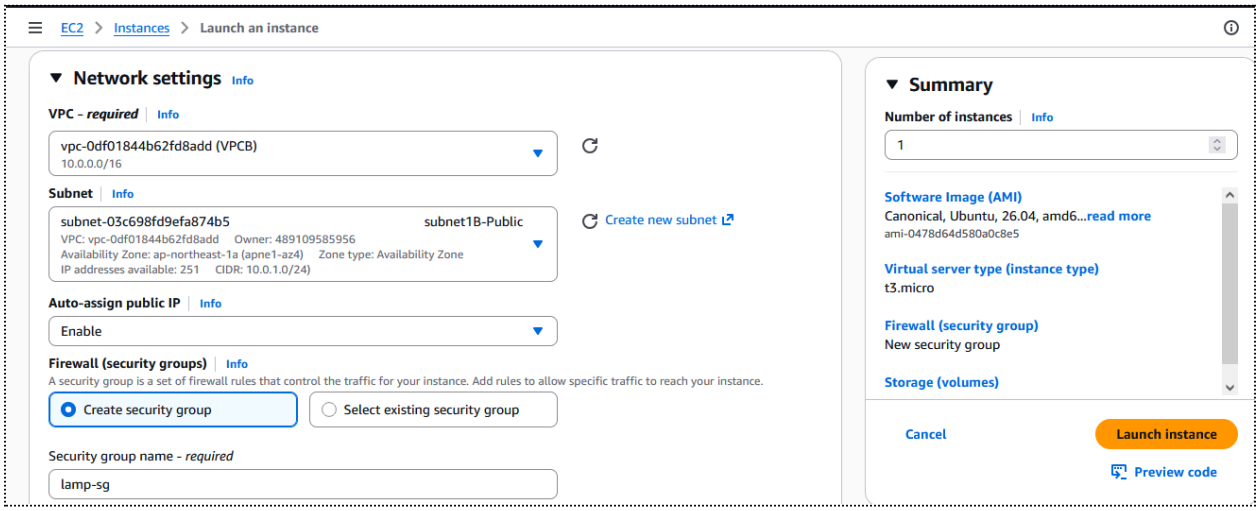
1. In VPCA (Mumbai region):

- Launch 1 EC2 instance name: EC2A
- Place it in private subnet (192.168.1.0/24)

2. In VPCB (Tokyo region):

- Launch 3 EC2 instances:

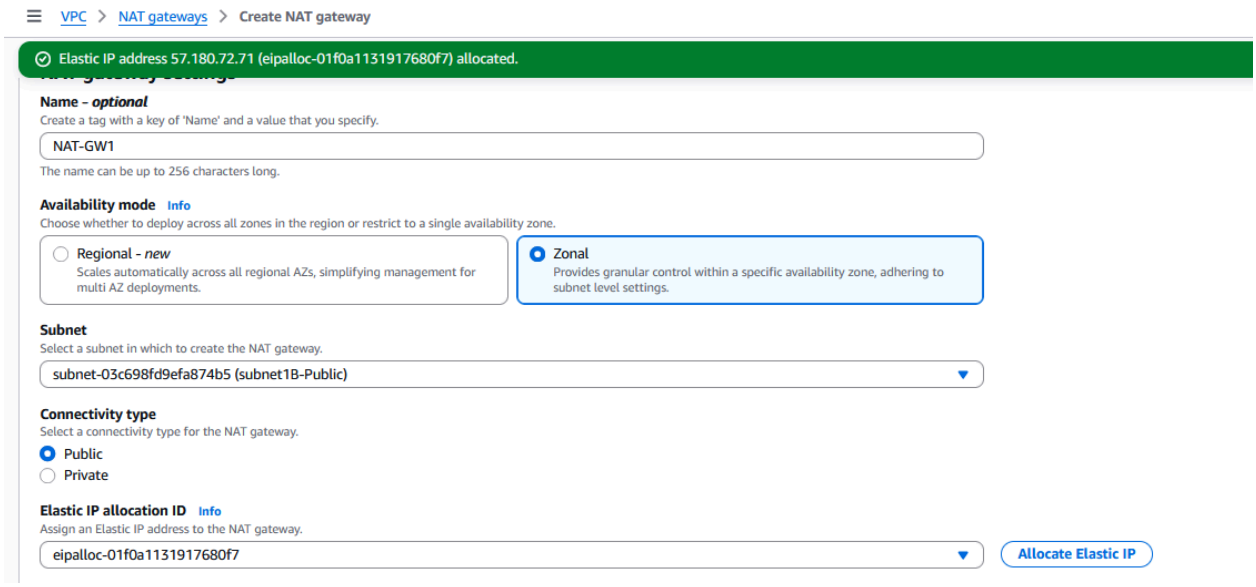
Subnet	EC2 Name
subnet1B-Public 10.0.1.0/24	EC2B1
subnet2B-Private 10.0.2.0/24	EC2B2
subnet3B-Private10.0.3.0/24	EC2B3

- 

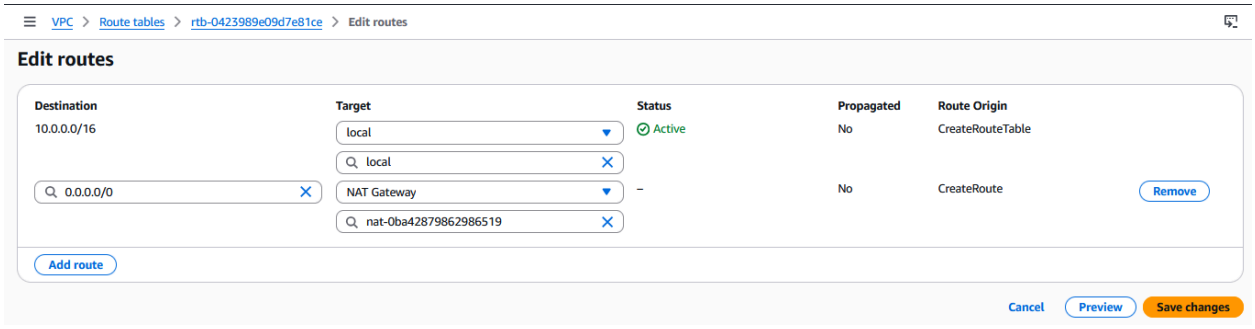
- NB: here I have used Ubuntu 26 for EC2B1 for Lamp and auto assign public should be enabled.

STEP 3: Create NAT Gateway

- ☐ Create NAT Gateway (name: NAT-GW1)in Subnet1B-Public (10.0.1.0/24) in the Tokyo region.

- [ ] 

- [ ] After creating NAT-GW1 edit Route table RTB3(associated with subnet3B-Private) and add NAT gateway

- [ ] 

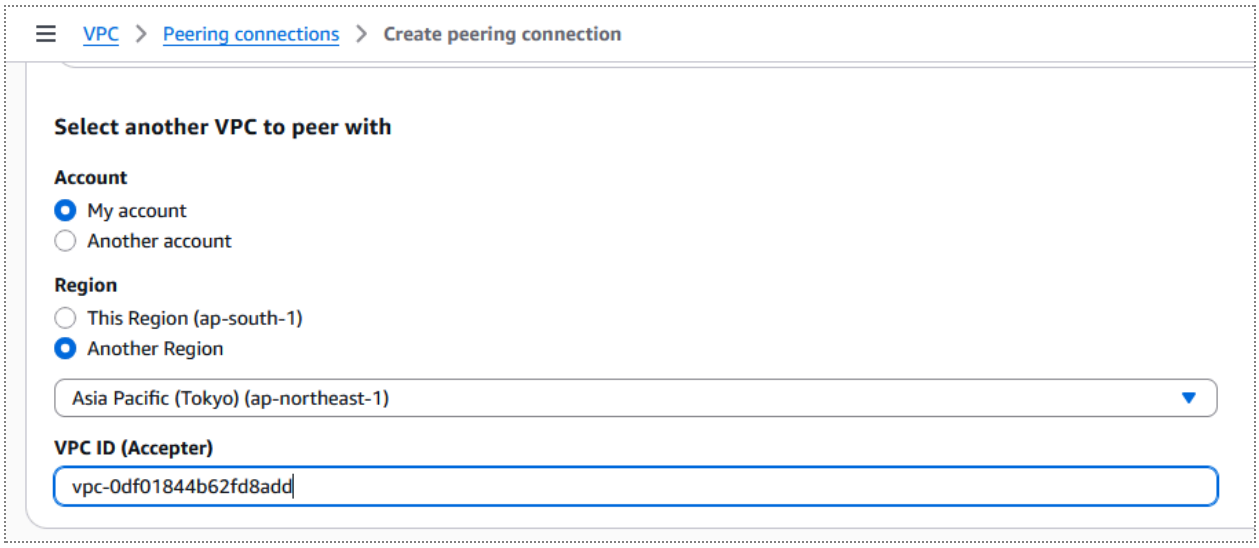
- [ ] Means Outbound internet allowed and Inbound internet blocked

Step 4: VPC Peering (Mumbai ↔ Tokyo)

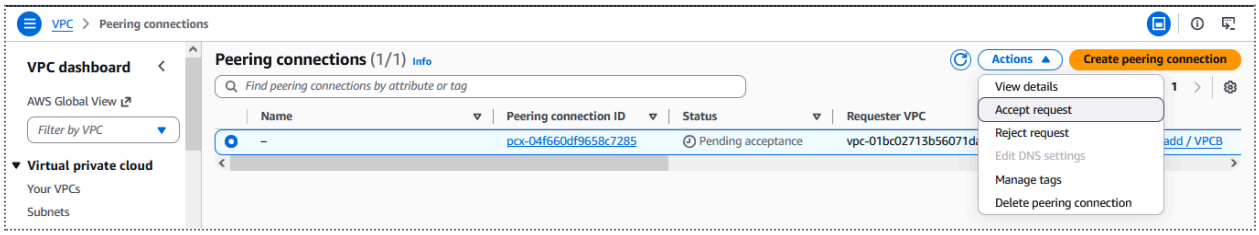
Create cross-region peering using Amazon VPC

4.1 Create Peering Connection

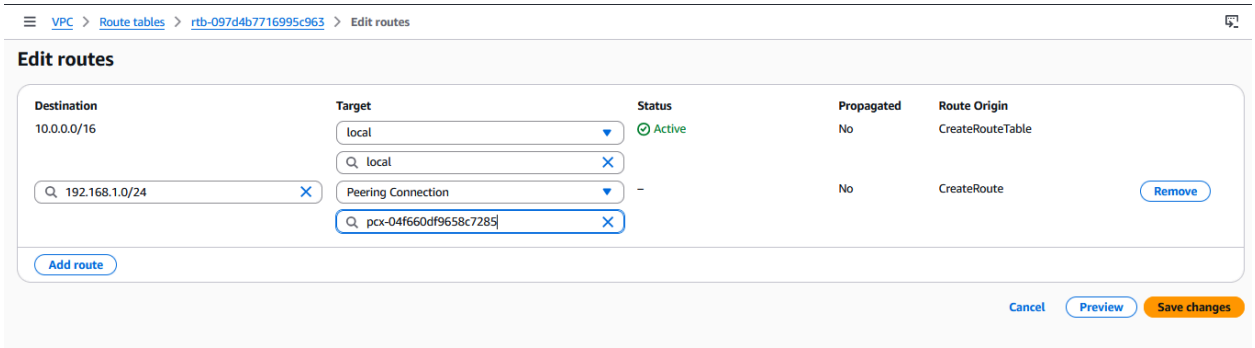
- Request peering:
    - VPC A (Mumbai)



- After creating peering request Status will be shown as *Pending Acceptance by ACCOUNTID*
- Accept requests from the VPCB, Tokyo region.

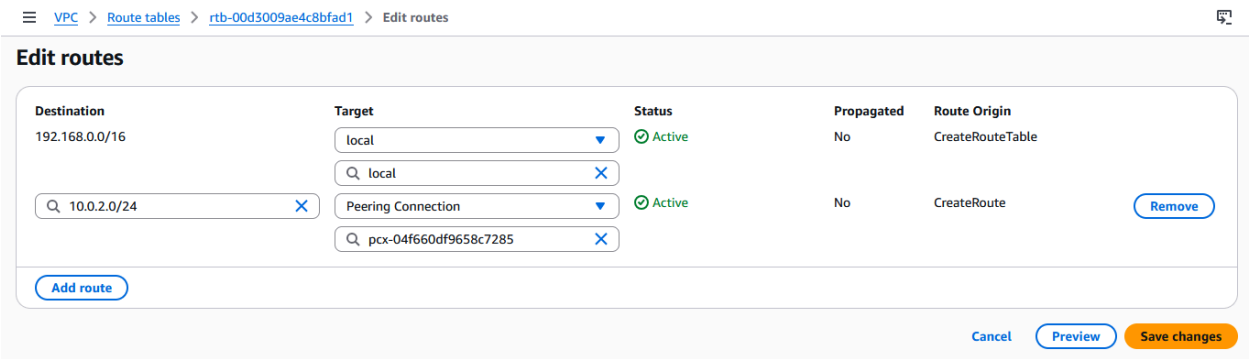


- Edit route of the route table of RTB2
- Add destination as Subnet1A-private CIDR



- Target peering connection and save changes.

- Go to Mumbai region and edit route table RTA1 add destination as subnet2B-Private



- And save changes

Step 5: Test Peering Connection

- From EC2B1 (VPCB) ssh to EC2B2
  - Copy the tokyo keypair private file to EC2B1 as key.pem and ssh
  - Chmod 400 key.pem
  - Ssh -i key.pem ec2-user@<privateip-of-EC2B2 instance>

```
ubuntu@ip-10-0-1-200:~$ vi key.pem
ubuntu@ip-10-0-1-200:~$ ssh -i key.pem ^C
ubuntu@ip-10-0-1-200:~$ chmod 400 key.pem
ubuntu@ip-10-0-1-200:~$ ssh -i key.pem ec2-user@10.0.2.131
The authenticity of host '10.0.2.131 (10.0.2.131)' can't be established.
ED25519 key fingerprint is: SHA256:E6E4R/RySoUz2O6GfdcBcyytVL4sgW8O9Tn7nwX6Sg4
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.131' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-2-131 ~]$
```

- Now you are in EC2B2. From there ping <privateip-of-EC2A-from VPCA>

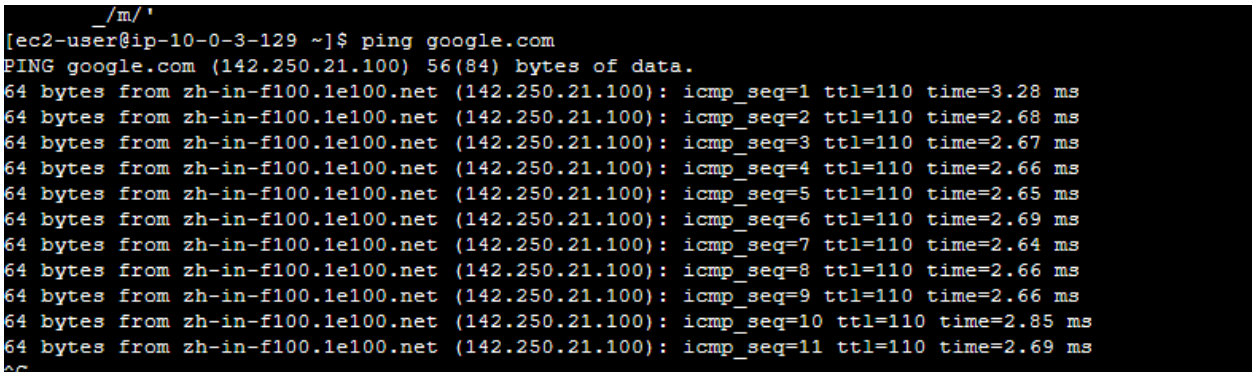
- If configured correctly:
  - Ping succeeds

```
ubuntu@ip-10-0-1-200:~$ ssh -i key.pem ec2-user@10.0.2.131
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu May 21 01:27:14 2026 from 10.0.1.200
[ec2-user@ip-10-0-2-131 ~]$ ping 192.168.1.157
PING 192.168.1.157 (192.168.1.157) 56(84) bytes of data.
64 bytes from 192.168.1.157: icmp_seq=1 ttl=127 time=127 ms
64 bytes from 192.168.1.157: icmp_seq=2 ttl=127 time=127 ms
64 bytes from 192.168.1.157: icmp_seq=3 ttl=127 time=127 ms
64 bytes from 192.168.1.157: icmp_seq=4 ttl=127 time=127 ms
```

- Private-to-private communication works via peering

Test NAT Gateway

SSH to EC2B3 from EC2B1 and ping google.com



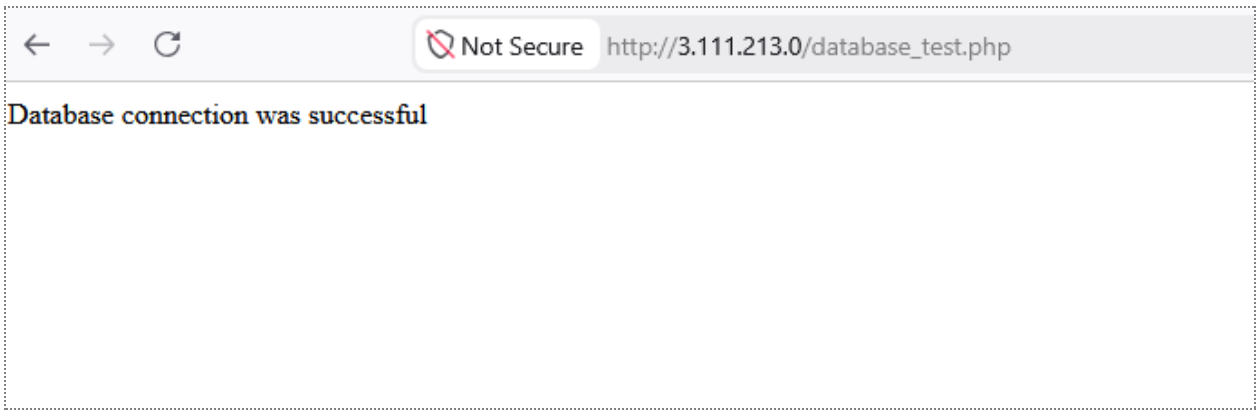
It works!

Step 6: LAMP Stack Setup (VPC B Public Subnet)

- Launch an EC2 instance in VPCB Public subnet named LAMP-EC2
- Ssh to LAMP-EC2 and Install LAMP stack
- Follow the link below for LAMP installation.
    - https://docs.vultr.com/install-linux-apache-mysql-and-php-lamp-on-ubuntu-20-04-lts
    - Then, visit the address below:

      http://<public ip of your instance>/database_test.php

    - You should get the output below that shows that the script has successfully connected to the database.



Conclusion

This project demonstrates a secure and scalable multi-region AWS architecture using VPCs in Mumbai and Tokyo. It includes proper subnet segmentation, EC2 deployment across public and private subnets, VPC peering for cross-region communication, and NAT Gateway for controlled internet access. Overall, it helps understand real-world cloud networking, security, and connectivity design in AWS.